



Chapter 19

Root Finding

By

Hyung Jae (Chris) Chang

Troy University



Introduction

- The root or zero of a function, $f(x)$: x_r such that $f(x_r) = 0$.
 - For some functions, it is clear finding the root of a function.
 $f(x) = x^2 - 9 \rightarrow$ the roots are 3 and -3
 - For some functions, determining an analytic or exact solution for the roots of functions can be difficult.
 $f(x) = \cos(x) - x$

TRY IT! Use the `fsolve` function from SciPy to compute the root of $f(x) = \cos(x) - x$ near -2. Verify that the solution is a root (or close enough).

```
In [1]: import numpy as np
        from scipy import optimize

        f = lambda x: np.cos(x) - x
        r = optimize.fsolve(f, -2)
        print("r =", r)

        # Verify the solution is a root
        result = f(r)
        print("result=", result)

r = [0.73908513]
result= [0.]
```



Introduction (cont.)

TRY IT! The function $f(x) = \frac{1}{x}$ has no root. Use the `fsolve` function to try to compute the root of $f(x) = \frac{1}{x}$. Turn on the `full_output` to see what is going on. Check the documentation for details.

```
In [2]: f = lambda x: 1/x

        r, infodict, ier, mesg =
            optimize.fsolve(f, -2, full_output=True)
        print("r =", r)

        result = f(r)
        print("result=", result)

        print(mesg)

r = [-3.52047359e+83]
result= [-2.84052692e-84]
The number of calls to function has reached maxfev = 400.
```

- The value `r` returned is not a root.
- In the code above, `full_output` option is used to see more information; “The number of calls to function has reached `maxfev=400`.”



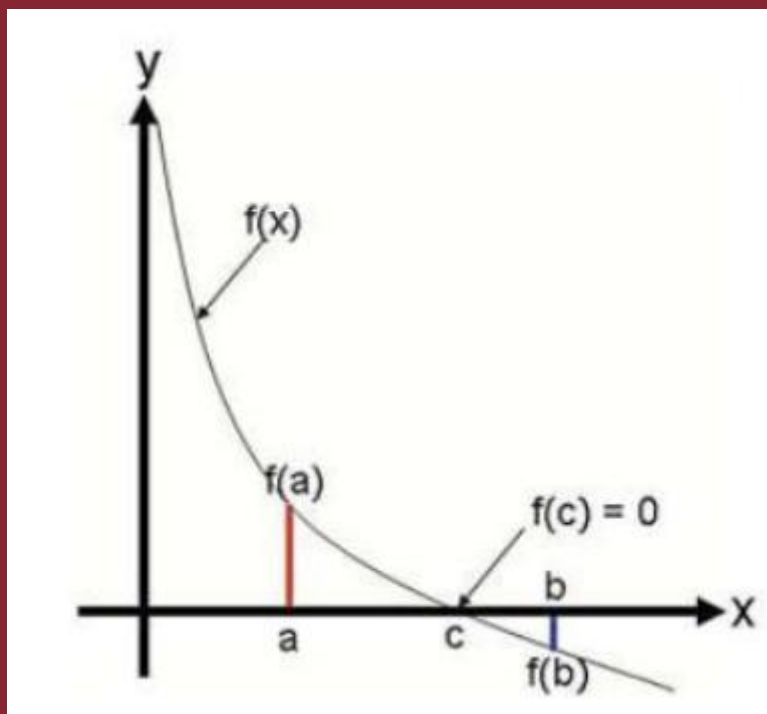
Tolerance

- **Error**: a deviation from an expected or computed value.
- **Tolerance**: the level of error that is acceptable for an engineering application.
- “Computer program has **converged** to a solution when it has found a solution with an error smaller than the tolerance.”
- For computing roots, we want an x_r such that $f(x_r)$ is very close to zero.



Bisection Method

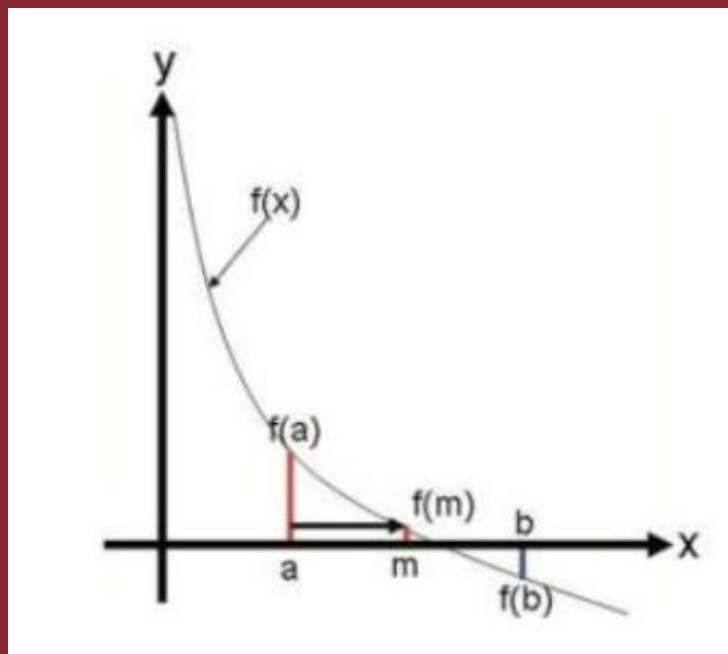
- **Intermediate Value Theorem:** if $f(x)$ is a continuous function between a and b , and $\text{sign}(f(a)) \neq \text{sign}(f(b))$, then there must be a " c ", such that $a < c < b$ and $f(c) = 0$.





Bisection Method (cont.)

- Bisection method uses the intermediate value theorem iteratively to find roots.
- By the intermediate value theorem, there must be a root in the open interval (a,b) under the following conditions.
 - Let $f(x)$ be a continuous function, and a and b be real scalar values such that $a < b$.
 - $f(a) > 0$ and $f(b) < 0$





Bisection Method in Python – Example 1

TRY IT! Program a function `my_bisection(f,a,b,tol)` that approximates a root r of f , bounded by a and b to within $|f(\frac{a+b}{2})| < \text{tol}$.

```
In [1]: import numpy as np

def my_bisection(f, a, b, tol):
    # approximates a root, R, of f bounded
    # by a and b to within tolerance
    # | f(m) | < tol with m being the midpoint
    # between a and b. Recursive implementation

    # check if a and b bound a root
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception(
            "The scalars a and b do not bound a root")

    # get midpoint
    m = (a + b)/2

    if np.abs(f(m)) < tol:
        # stopping condition, report m as root
        return m
    elif np.sign(f(a)) == np.sign(f(m)):
        # case where m is an improvement on a.
        # Make recursive call with a = m
        return my_bisection(f, m, b, tol)
    elif np.sign(f(b)) == np.sign(f(m)):
        # case where m is an improvement on b.
        # Make recursive call with b = m
        return my_bisection(f, a, m, tol)
```



Bisection Method in Python – Example 2

TRY IT! The $\sqrt{2}$ can be computed as the root of the function $f(x) = x^2 - 2$. Starting at $a = 0$ and $b = 2$, use `my_bisection` to approximate the $\sqrt{2}$ to a tolerance of $|f(x)| < 0.1$ and $|f(x)| < 0.01$. Verify that the results are close to a root by plugging the root back into the function.

```
In [2]: f = lambda x: x**2 - 2

        r1 = my_bisection(f, 0, 2, 0.1)
        print("r1 =", r1)
        r01 = my_bisection(f, 0, 2, 0.01)
        print("r01 =", r01)

        print("f(r1) =", f(r1))
        print("f(r01) =", f(r01))
```

```
r1 = 1.4375
r01 = 1.4140625
f(r1) = 0.06640625
f(r01) = -0.00042724609375
```



Newton-Raphson Method

- Let $f(x)$ be a smooth function and x_r be an unknown root of $f(x)$.
- Assume that x_0 is a guess for $x_r \rightarrow$ most likely, x_0 will not be a root.
- Based on this, we want to find x_1 which is an improved guess from x_0 .
- If x_0 is “close enough” to x_r , then we improve upon it by taking the linear approximation of $f(x)$ around x_0 which is a line and finding the intersection of this line with the x -axis.
- The linear approximation of $f(x)$ around x_0

$$0 = f(x_0) + f'(x_0)(x_1 - x_0)$$



$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$



Newton-Raphson Method (cont.)

- A Newton step computes an improved guess x_i using the previous guess x_{i-1} .

$$x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$$

- The Newton-Raphson Method of finding roots iterates Newton steps from x_0 until the error is less than the tolerance.



Example 1

TRY IT! Again, the $\sqrt{2}$ is the root of the function $f(x) = x^2 - 2$. Using $x_0 = 1.4$ as a starting point, use the previous equation to estimate $\sqrt{2}$. Compare this approximation with the value computed by Python's `sqrt` function.

$$x = 1.4 - \frac{1.4^2 - 2}{2(1.4)} = 1.4142857142857144.$$

```
In [1]: import numpy as np

f = lambda x: x**2 - 2
f_prime = lambda x: 2*x
newton_raphson = 1.4 - (f(1.4))/(f_prime(1.4))

print("newton_raphson =", newton_raphson)
print("sqrt(2) =", np.sqrt(2))
```

```
newton_raphson = 1.4142857142857144
sqrt(2) = 1.4142135623730951
```



Example 2

TRY IT! Write a function `my_newton(f, df, x0, tol)` where the output is an estimate of the root of f , f is a function object $f(x)$, df is a function object $f'(x)$, $x0$ is an initial guess, and tol is the error tolerance. The error measurement should be $|f(x)|$.

```
In [2]: def my_newton(f, df, x0, tol):  
        # output is an estimation of the root of f  
        # using the Newton-Raphson method  
        # recursive implementation  
        if abs(f(x0)) < tol:  
            return x0  
        else:  
            return my_newton(f, df, x0 - f(x0)/df(x0), tol)
```



Example 3

TRY IT! Use `my_newton` to compute $\sqrt{2}$ to within a tolerance of $1e-6$ starting at $x_0 = 1.5$.

```
In [3]: estimate = my_newton(f, f_prime, 1.5, 1e-6)
        print("estimate =", estimate)
        print("sqrt(2) =", np.sqrt(2))
```

```
estimate = 1.4142135623746899
sqrt(2) = 1.4142135623730951
```



Limitation of Newton-Raphson Method

- Then, how do we make an initial guess which is close enough to the root? → impossible unless we have some special information about the function known a priori (e.g. “the function has a root close to $x=0$ ”.)
- If the derivative at a guess is close to zero, then the Newton step will be very large and probably lead far away from the root.
- Depending on the behavior of the function derivative between x_0 and x_r , the Newton-Raphson method may converge to a different root than x_r which may not be useful for the engineering application being considered.



Limitation of Newton-Raphson Method (cont.)

TRY IT! Compute a single Newton step to get an improved approximation of the root of the function $f(x) = x^3 + 3x^2 - 2x - 5$ and initial guess $x_0 = 0.29$.

```
In [4]: x0 = 0.29
        x1 = x0 - (x0**3 + 3*x0**2 - 2*x0 - 5) / (3*x0**2 + 6*x0 - 2)
        print("x1 =", x1)

x1 = -688.4516883116648
```

Note that $f'(x_0) = -0.0077$ (is close to zero), and the error at x_1 is approximately 324880000 (very large).

TRY IT! Consider the polynomial $f(x) = x^3 - 100x^2 - x + 100$. This polynomial has a root at $x = 1$ and $x = 100$. Use the Newton-Raphson method to find a root of f starting at $x_0 = 0$.

At $x_0 = 0$, $f(x_0) = 100$, and $f'(x) = -1$, the Newton step gives $x_1 = 0 - \frac{100}{-1} = 100$, which is a root of f . Note that this root is much farther from the initial guess than the other root at $x = 1$, and it may not be the root you wanted from an initial guess of zero.



Root Finding in Python

- Python has root-finding functions: `f_solve` function from `scipy.optimize`.
- `f_solve` function takes in many arguments among which the most important two are
 - The function that you want to find the root
 - The initial guess

TRY IT! Compute the root of the function $f(x) = x^3 - 100x^2 - x + 100$ using `f_solve`.

```
In [1]: from scipy.optimize import fsolve
```

```
In [2]: f = lambda x: x**3-100*x**2-x+100
```

```
        fsolve(f, [2, 80])
```

```
Out[2]: array([ 1., 100.])
```